The Virtual Store – E- Commerce Website

Project Report (Proj -CS- 402 (Project -III))

Submitted for the

Bachelor of Technology

In

Computer Science And Engineering

Submitted By

Divyansh

252102072

Anurag

252102074

Under the Supervision of

Mrs. Jyoti Tamak

Faculty of CSE

Deptt. of CSE,

UIET, KUK

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

KURUKSHETRA UNIVERSITY, KURUKSHETRA

May 2025

Certificate

This is to certify that the project entitled **"**The Virtual Store – E- Commerce Website**"** submitted by Divyansh, 252102072 and Anurag, 252102074 to the Department of Computer Science and Engineering of University Institute of Engineering and Technology, Kurukshetra University, Kurukshetra for the Bachelor of Technology in Computer Science and Engineering, is a bonafide research work carried out by them under my supervision and guidance. They have reached the standard of fulfilling the requirements of regulations relating to the degree.

I wish them success in all of their future endeavours.

(Mrs. Jyoti Tamak)
Faculty of CSE Deptt.
UIET, KUK

Candidates' Declaration

We hereby declare that the project entitled "The Virtual Store – E- Commerce Website", which is being submitted for Bachelor of Technology in Computer Science and Engineering. is an authentic record of genuine work done under the supervision of Mrs. Jyoti Tamak, Faculty of Computer Science & Engineering, University Institute of Engineering and Technology, Kurukshetra. The matter in this project has not been submitted for the award of any degree/diploma of this or any other University/Institute.

Date:

Students' Name:
Divyansh
252102072

Anurag
252102074

Project Topic

The Virtual Store – E- Commerce Website

Abstract:

The Virtual Store is an innovative e-commerce platform dedicated to providing a curated selection of urban fashion apparel and accessories for fashion-forward individuals. In a rapidly evolving fashion landscape, Urban Wardrobe aims to bridge the gap between trendy urban styles and accessibility, offering customers a seamless online shopping experience.

This e-commerce website is designed to cater to the diverse tastes and preferences of urban fashion enthusiasts, showcasing a wide range of clothing, footwear, and accessories sourced from leading urban fashion brands. With a focus on quality, style, and affordability, Urban Wardrobe strives to become the go-to destination for those seeking the latest trends in urban fashion.

The website's user-friendly interface allows customers to easily browse through various categories, explore new arrivals, and discover exclusive collections. Detailed product descriptions, along with high-resolution images, ensure that shoppers can make informed purchasing decisions. Additionally, secure payment gateways and streamlined checkout processes provide a safe and convenient shopping experience.

Table Of Content

- **1.0 - Introduction** **1**
  - 1.1 -Objective
  - 1.2 - Methodology Used
- **2.0 - Technology Stack Used** **5**
  - 2.1 - Front End
  - 2.2 - Backend
  - 2.3 -Database
  - 2.4 - Three Tier Architecture
- **3.0 - Week 1: Project Setup** **10**
- **4.0 - Week 2 & 3: Frontend Development** **12**
- **5.0 - WEEK 4 & 5: Backend Development** **14**
- **6.0 - WEEK 6 & 7: Website Testing** **17**
- **7.0 - Output** **20**
- **8.0 - Conclusion & 8.1 - Future Modifications** **23**
- **9.0 - Appendix** **27**

1.0 - Introduction

In the pulsating heart of contemporary style lies The Virtual Store —a digital sanctuary meticulously crafted for the urban fashion aficionado. Welcome to a realm where urban culture meets the convenience of online shopping, where every click unlocks a treasure trove of trendsetting attire and accessories.

Urban Wardrobe emerges from a fusion of passion and purpose, driven by the desire to redefine how urban fashion is experienced and accessed. Our e-commerce project represents a dynamic intersection of style, technology, and community—a platform where fashion-forward individuals can explore, express, and elevate their personal style effortlessly.

Step into our virtual boutique, and you'll encounter an eclectic array of urban fashion essentials curated to inspire and empower. From street-chic ensembles to avant-garde statements, our collection embodies the raw energy and unapologetic spirit of urban culture, curated with an unwavering commitment to quality and authenticity.

More than just an online store, Urban Wardrobe is a beacon of inclusivity and creativity—a digital melting pot where diverse voices converge to celebrate the kaleidoscope of urban fashion. Whether you're a trendsetter pushing boundaries or a style seeker on a quest for inspiration, our platform is your canvas, your playground, your sanctuary.

Join us as we embark on this exhilarating journey through the vibrant landscape of urban fashion. At Urban Wardrobe, the streets are our runway, and your style is our priority. Welcome to a world where possibilities are endless, and the pursuit of style knows no bounds. Welcome to Urban Wardrobe—where urban fashion meets online convenience, and your fashion journey begins anew.

1.1 - Objective:

Curate Trendsetting Collections: To consistently curate and offer trendsetting urban fashion collections that resonate with the style preferences of our target audience.

Enhance User Experience: To continually enhance the user experience on our e-commerce platform, ensuring easy navigation, intuitive design, and seamless transactions.

Expand Product Range: To expand our product range to include a diverse selection of clothing, footwear, accessories, and lifestyle products that cater to the evolving needs and tastes of our customers.

Build Brand Awareness: To increase brand awareness and establish Urban Wardrobe as a reputable and trusted authority in the urban fashion industry through strategic marketing initiatives and brand-building efforts.

Foster Community Engagement: To foster a vibrant and engaged community of fashion enthusiasts by providing opportunities for interaction, collaboration, and user-generated content sharing on our platform and social media channels.

Optimize Operations: To optimize e-commerce operations and logistics processes to ensure timely delivery, efficient inventory management, and excellent customer service.

Drive Sales and Revenue Growth: To drive sales and revenue growth by implementing effective promotional campaigns, exploring strategic partnerships, and maximizing conversion rates through targeted marketing efforts.

Embrace Sustainability: To embrace sustainability practices within our operations and supply chain, by sourcing eco-friendly materials, reducing waste, and promoting ethical manufacturing practices wherever possible.

Personalize Customer Experience: To personalize the customer experience through data-driven insights, tailored recommendations, and personalized communication, thereby fostering long-term customer loyalty and retention.

Stay Ahead of Industry Trends: To stay ahead of industry trends and technological advancements in e-commerce and fashion retailing, continuously innovating and adapting our strategies to remain competitive in the market.

These objectives can serve as guiding principles for Urban Wardrobe's growth and development, helping to steer the company towards long-term success in the urban fashion landscape.

1.2 - Methodology Used:

Market Research: Conduct comprehensive market research to understand the dynamics of the urban fashion industry, including consumer preferences, emerging trends, competitor analysis, and market opportunities.

Target Audience Profiling: Define and segment the target audience based on demographics, psychographics, and behavior patterns to tailor products and marketing strategies accordingly.

Product Curation and Sourcing: Curate a diverse range of urban fashion products, including clothing, footwear, and accessories, sourced from reputable brands and suppliers known for quality and authenticity.

E-commerce Platform Selection: Choose a suitable e-commerce platform based on factors such as scalability, customization options, user interface, and integration capabilities to support Urban Wardrobe's business objectives.

Website Development and Design: Develop and design the e-commerce website with a focus on user experience, aesthetics, and functionality, ensuring easy navigation, mobile responsiveness, and secure payment gateways.

Content Creation and Marketing: Create engaging and visually appealing content, including product descriptions, imagery, videos, and blog posts, to showcase the products and connect with the target audience through various digital marketing channels such as social media, email marketing, and influencer collaborations.

Community Building: Foster a sense of community and engagement among customers by encouraging user-generated content, hosting interactive events, and facilitating discussions on fashion-related topics to build brand loyalty and advocacy.

Data Analytics and Optimization: Implement data analytics tools to track website performance, customer behaviour, sales metrics, and marketing effectiveness, using insights to optimize strategies, improve decision-making, and enhance the overall customer experience.

Customer Service and Satisfaction: Prioritize exceptional customer service by providing prompt assistance, easy returns, and personalized recommendations, aiming to exceed customer expectations and build long-term relationships.

Continuous Improvement: Continuously evaluate and iterate on strategies, processes, and offerings based on customer feedback, market trends, and business goals, striving for continuous improvement and innovation to stay competitive in the dynamic urban fashion landscape.

2.0 - Technology Stack Used

2.1 - Front End:

1. HTML (Hyper Text Markup Language): HTML is the standard markup language for creating the structure of web pages.

2. CSS (Cascading Style Sheets): CSS is used for styling HTML elements, controlling layout, colors, fonts, and other visual aspects of the website.

3. JavaScript: JavaScript is a programming language that adds interactivity and dynamic behavior to web pages. It's commonly used for client-side scripting, such as form validation, animations, and interactive elements.

Frontend Frameworks:

React.js: React.js is a popular JavaScript library for building user interfaces, known for its component-based architecture and virtual DOM for efficient rendering. It's well-suited for building dynamic and responsive web applications.

1.HTML (Hyper Text Markup Language):

Advantages:

Standardized markup language for creating the structure of web pages.
Easy to learn and widely supported across browsers and devices.

Disadvantages:

Limited in terms of interactivity and dynamic behavior without the use of additional scripting languages like JavaScript.

2.CSS (Cascading Style Sheets):

Advantages:

Allows for precise control over the visual presentation of web pages, including layout, colors, fonts, and animations.

Can be easily integrated with HTML to style elements consistently across a website.

Disadvantages:

Cascading nature can lead to unintended style conflicts and specificity issues.

Complex layouts and designs may require extensive CSS code, leading to maintenance challenges.

3.JavaScript:

Advantages:

Enables interactivity, dynamic updates, and client-side processing within web pages.

Wide range of libraries, frameworks, and tools available for building complex web applications.

Can be used for both frontend and backend development (Node.js).

Disadvantages:

Browser compatibility issues can arise, requiring careful testing and fallback mechanisms.

Overuse of JavaScript can impact page load times and performance, especially on mobile devices.

Frontend Frameworks:

4.React.js:

Advantages:

Component-based architecture promotes code reusability and maintainability.

Virtual DOM enables efficient rendering and performance optimization.

Disadvantages:

Steeper learning curve for beginners, especially with advanced concepts like JSX and state management.

Requires additional setup and configuration compared to simpler libraries.

2.2 - Backend:

Node.js for Server:

Node.js is an open-source, cross-platform runtime environment built on JavaScript's Chrome V8 engine. It's ideal for event-driven, real-time applications.

Advantages:

Asynchronous Programming: Node.js excels at handling multiple concurrent requests efficiently.

JavaScript Familiarity: Developers familiar with JavaScript can easily pick up Node.js.

Large Community and Ecosystem: Node.js benefits from a vast community and a rich ecosystem of libraries and tools.

Reasons to Use:

Faster Development Cycles: Node.js' hot reloading feature allows developers to see changes instantly within the running application, accelerating development iterations.

Microservices Architecture: Node.js is well-suited for building modular, microservice-based architectures, making applications more scalable and easier to maintain.

Large Talent Pool: The growing popularity of Node.js means there's a readily available pool of developers familiar with the technology.

MongoDB for Database:

MongoDB is a NoSQL document database that stores data in flexible JSON-like documents.

Advantages:

Schema-less: MongoDB eliminates the need for predefined schemas, offering flexibility for evolving data models.

Performance: MongoDB is known for its fast performance, particularly for unstructured data.

Ease of Use: MongoDB's query language is similar to JavaScript, making it easy to learn and use.

Reasons to Use:

Horizontal Scalability: MongoDB can easily scale horizontally by adding additional servers to handle increasing data volumes.

Rich Query Language: MongoDB's query language (MQL) offers advanced capabilities for querying and filtering complex JSON documents.

2.3 - Three Tier Architecture

A three-tier architecture for a website is a design pattern that divides the application into three logical layers: presentation layer (frontend), application layer (backend), and data layer (database). This architecture provides modularity, scalability, and maintainability by separating concerns and allowing each layer to be developed, deployed, and managed independently. Here's an overview of each tier:

Presentation Layer (Frontend):

The presentation layer, also known as the frontend, is responsible for handling the user interface and user experience of the website.
It typically consists of web pages, user interfaces, and client-side code that interact with users through browsers or other client devices.
Technologies commonly used in the presentation layer include HTML, CSS, JavaScript, and frontend frameworks like React.js, Vue.js, or Angular.
The presentation layer communicates with the application layer (backend) through HTTP requests and responses, consuming data and services provided by the backend.

Application Layer (Backend):

The application layer, also known as the backend, serves as the business logic and processing engine of the website.
It handles user requests, performs data processing, executes business rules, and interacts with the data layer to retrieve or store data.
The backend is responsible for managing user authentication, authorization, session management, and other application-level functionalities.
Technologies commonly used in the application layer include server-side programming languages (e.g., Node.js, Python, Ruby), web frameworks (e.g., Express.js, Django, Ruby on Rails), and middleware for handling HTTP requests (e.g., Express.js middleware).
The application layer communicates with the presentation layer (frontend) through APIs (Application Programming Interfaces), serving data and services in a format that can be consumed by the frontend.

Data Layer (Database):

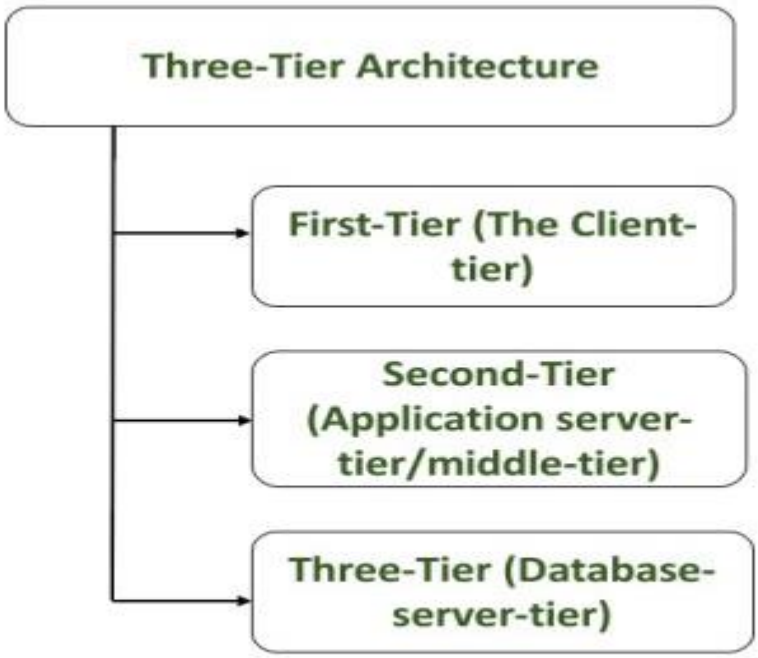
The data layer, also known as the backend database, is responsible for storing, retrieving, and managing the application's data.

It consists of one or more databases that store structured or unstructured data, depending on the requirements of the application.

Common types of databases used in the data layer include relational databases (e.g., PostgreSQL, MySQL) and NoSQL databases (e.g., MongoDB, Redis).

The data layer provides data persistence and data integrity, ensuring that data is stored securely and efficiently.

The application layer (backend) interacts with the data layer through database queries and transactions, accessing and manipulating data as needed to fulfill user requests.



3.0 - Week 1: Project Setup

During the initial setup phase of a basic web project, there are several key steps to follow to establish a solid foundation. Here's a detailed guide:

Project Planning:

- Define project goals, objectives, and requirements.
- Identify target audience and user personas.
- Create a project timeline and allocate resources accordingly.

Choose a Tech Stack:

Select appropriate technologies for the frontend, backend, and database layers based on project requirements, team expertise, and scalability considerations.
For example, for the frontend, you might choose HTML, CSS, JavaScript, and React.js; for the backend, you might choose Node.js with Express.js; and for the database, you might choose MongoDB.

Set Up Development Environment:

Install necessary development tools and software (e.g., VS code version control system like, package manager like npm).
Create a project directory structure to organize files and assets.

Frontend Setup:

Create HTML templates for the website's layout and structure.
Style the website using CSS to enhance visual appearance and user experience.
Implement client-side interactivity and functionality using JavaScript and frontend frameworks/libraries React.js.

Backend Setup:

Initialize a backend server using the chosen backend technology (e.g., Node.js with Express.js).
Set up routes and handlers to handle HTTP requests and responses.
Implement basic server-side logic, such as user authentication, data validation, and error handling.

Database Setup:

Install and configure the chosen database system (e.g., MongoDB).
Create database schemas and models to define data structures and relationships.
Set up database connections and perform initial data seeding if necessary**.**

4.0 - Week 2 & 3: Frontend Development

Project Setup:

- Choose a code editor or IDE (Integrated Development Environment) for coding.
- Set up a version control system and initialize a repository for your project.
- Decide on a package manager (e.g., npm) for managing dependencies.
- Create a project directory structure to organize your files.

Select a Tech Stack:

- Choose frontend technologies based on project requirements, such as HTML, CSS, JavaScript, and any relevant frameworks or libraries e.g., React.js.

HTML Structure:

- Set up the basic structure of your web pages using HTML (Hypertext Markup Language).
- Create semantic HTML markup for elements such as headers, navigation menus, content sections, forms, and footers.

CSS Styling:

- Style the HTML elements using CSS (Cascading Style Sheets) to enhance the visual appearance and layout of the web pages.
- Apply CSS properties for colours, typography, spacing, borders, backgrounds, and responsive design.

Frontend Frameworks and Libraries:

- If using a frontend framework or library (e.g., React.js), set up the development environment and project scaffolding according to the framework's documentation.

Browser Compatibility:

- Test your website or application across different web browsers (e.g., Chrome) to ensure compatibility and consistent rendering.
- Use feature detection and progressive enhancement techniques to provide fallbacks for unsupported browser features.

Performance Optimization:

- Optimize the performance of your frontend code to improve page load times and user experience.
- Minify and bundle CSS and JavaScript files, optimize images, leverage browser caching, and reduce HTTP requests.

Documentation:

- Document your frontend code, including HTML structure, CSS styles, JavaScript functions, and component usage.
- Provide clear and concise comments, README files, and documentation to aid developers and stakeholders in understanding and maintaining the codebase.

Continuous Improvement:

- Continuously monitor and analyze frontend performance, user feedback, and analytics data to identify areas for improvement.
- Iterate on your frontend codebase, implementing new features, fixing bugs, and refining the user experience based on feedback and insights.
- By following these steps, you can effectively plan, develop, test, and deploy frontend code for your web projects, ensuring a high-quality user experience and seamless interaction with your website or application.

5.0 - Week 4 & 5: Backend Development

Developing the backend for an e-commerce website involves various steps to ensure functionality, security, and scalability. Here's a comprehensive guide:

Requirements Gathering:

- Define the functional requirements of the e-commerce website, including user authentication, product management, shopping cart functionality, order processing, payment integration, and administrative features.
- Identify non-functional requirements such as performance, scalability, security, and compliance with industry standards and regulations (e.g., PCI DSS for).

Choose a Tech Stack:

- Select backend technologies based on project requirements, scalability needs, and team expertise. Common choices include programming languages like Node.js, Python, Ruby, or Java, along with frameworks and libraries such as Express.js, Django, Ruby on Rails, or Spring Boot.
- Choose a database system that suits the requirements of the application, such as relational databases like PostgreSQL or MySQL, or NoSQL databases like MongoDB or Redis.

Database Design:

- Design the database schema to represent entities such as users, products, orders, and transactions.
- Define relationships between entities using foreign keys and establish data integrity constraints.
- Optimize the database schema for performance and scalability, considering factors such as indexing, denormalization, and data partitioning.

Backend Architecture:

- Define the architecture of the backend application, considering factors such as modularity, scalability, and maintainability.
- Choose an appropriate architectural pattern (e.g., MVC, RESTful API) to organize the codebase and separate concerns such as business logic, data access, and presentation.

Set Up Development Environment:

- Install necessary development tools and software, including a code editor or IDE, version control system (e.g., Git), and dependencies management tools (e.g., npm, pip).
- Create a project directory structure to organize backend code, configuration files, and resources.

Backend Implementation:

- Implement user authentication and authorization mechanisms, including registration, login, logout, password hashing, and access control.
- Develop APIs (Application Programming Interfaces) to expose backend functionality for interacting with the frontend and other services. Define endpoints for operations such as fetching product listings, adding items to the shopping cart, processing orders, and managing user accounts.
- Implement business logic for handling e-commerce operations, such as inventory management, pricing, discounts, promotions, and tax calculations.
- Integrate third-party services and APIs for features like payment processing, shipping, and order tracking.

Security Implementation:

- Implement security best practices to protect the backend against common vulnerabilities such as SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), and unauthorized access.
- Use parameterized queries, input validation, and output encoding to prevent injection attacks.
- Implement HTTPS (SSL/TLS) for secure communication between clients and the server, and configure CORS (Cross-Origin Resource Sharing) policies to restrict cross-origin requests.

Documentation:

- Document the backend codebase, including API endpoints, request and response formats, authentication mechanisms, error handling, and data models.
- Provide clear and comprehensive documentation to aid developers, testers, and system administrators in understanding and using the backend system effectively.

Monitoring and Maintenance:

- Implement monitoring and logging solutions to track backend performance, errors, and system health indicators.
- Set up alerts and notifications for critical events and performance anomalies, and conduct regular audits and security scans to identify and mitigate vulnerabilities.
- Establish a maintenance plan for routine updates, patches, and bug fixes, and monitor user feedback and analytics data to prioritize feature enhancements and optimizations.

By following these steps, we can develop a robust and scalable backend for your e-commerce website, ensuring secure and efficient operation to support your business goals and meet the needs of your users.

6.0 - Week 6 &7 Website Testing

Website testing is crucial to ensure the quality, functionality, and performance of your web application. Here are the steps involved in comprehensive website testing:

Requirement Analysis:

- Understand the project requirements, including functional and non-functional aspects.
- Review user stories, wireframes, and design documents to identify test scenarios.

Test Planning:

- Develop a test plan outlining the testing approach, scope, objectives, resources, and schedule.
- Identify testing types (e.g., functional testing, usability testing, performance testing) and prioritize based on project goals.

Test Environment Setup:

- Set up test environments mirroring production, including hardware, software, browsers, and network configurations.
- Configure test databases, servers, and other dependencies required for testing.

Test Case Design:

- Create test cases based on requirements, user stories, and use cases.
- Define test scenarios covering positive, negative, and edge cases to validate functionality, usability, and security.
- Include test data, expected results, and preconditions for each test case.

Functional Testing:

- Perform functional testing to validate that the website behaves as expected and meets functional requirements.
- Execute test cases to verify features such as navigation, forms, links, buttons, user authentication, and data validation.
- Use techniques like equivalence partitioning, boundary value analysis, and decision tables to design effective test cases.

Usability Testing:

- Conduct usability testing to evaluate the user experience and interface design of the website.
- Gather feedback from target users to assess ease of use, navigation, layout, readability, and accessibility.
- Identify areas for improvement in design, content, and interaction flow to enhance user satisfaction.

Compatibility Testing:

- Perform compatibility testing to ensure that the website functions correctly across different browsers, devices, and operating systems.
- Test the website on popular web browsers (e.g., Chrome, Firefox, Safari, Edge) and various devices (e.g., desktops, tablets, smartphones).
- Check for responsiveness, layout consistency, and feature compatibility across different screen sizes and resolutions.

Performance Testing:

- Conduct performance testing to evaluate the website's responsiveness, scalability, and load handling capabilities.
- Perform load testing, stress testing, and endurance testing to assess system behaviour under different levels of traffic and usage.

- Measure response times, throughput, resource utilization, and server-side metrics to identify performance bottlenecks and optimize system performance.

Security Testing:

- Perform security testing to identify vulnerabilities and protect against potential threats and attacks.
- Conduct vulnerability assessments, penetration testing, and code reviews to uncover security weaknesses in the website and associated systems.
- Verify security controls such as encryption, authentication, authorization, session management, and data protection mechanisms.

Regression Testing:

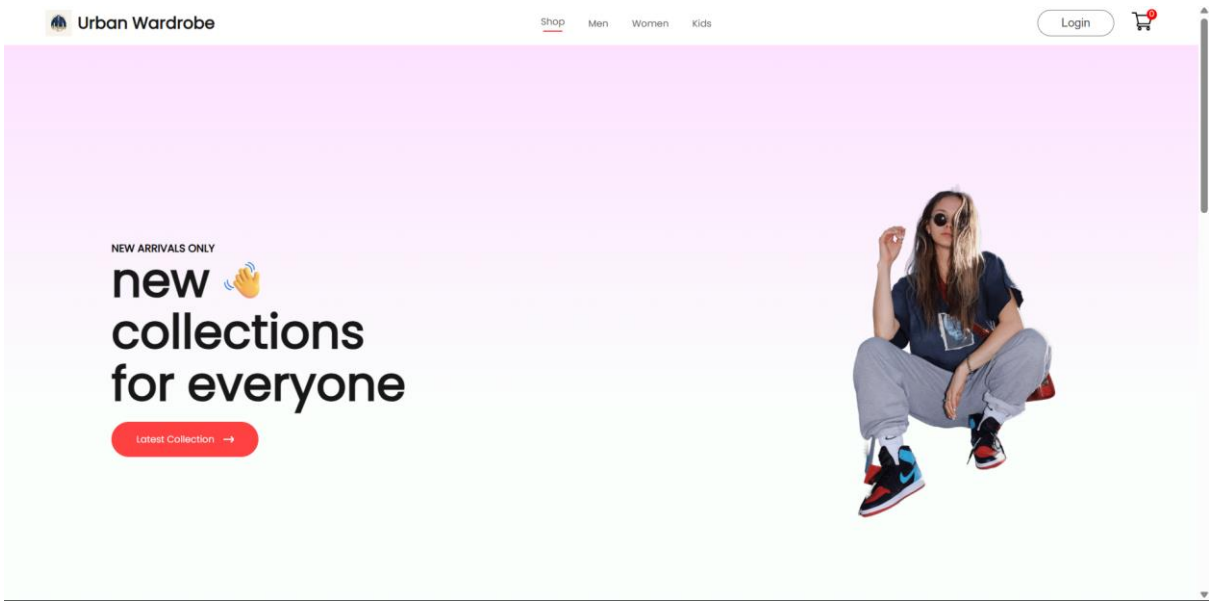
- Execute regression testing to ensure that recent changes or updates have not introduced new defects or impacted existing functionality.
- Re-run previously executed test cases and automated tests to validate that no regression issues have occurred.
- Prioritize test cases based on risk and impact to focus testing efforts on critical areas of the application.

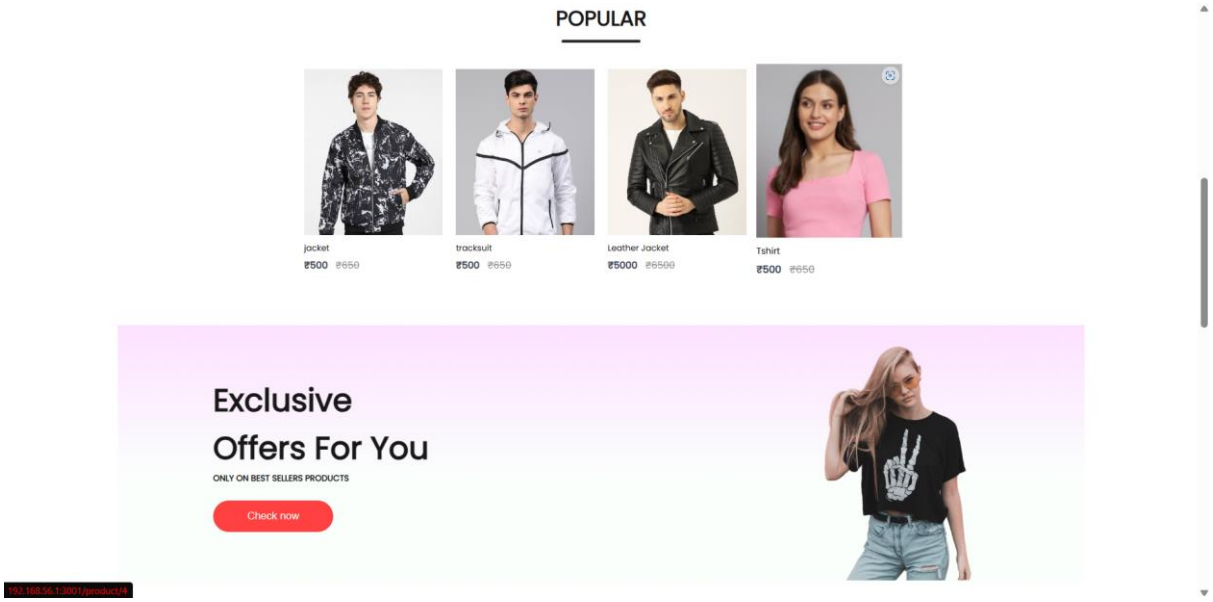
Accessibility Testing:

- Perform accessibility testing to ensure that the website is usable and accessible to users with disabilities.
- Evaluate compliance with accessibility standards such as WCAG (Web Content Accessibility Guidelines) to address issues related to screen readers, keyboard navigation, colour contrast, and alternative text for images.

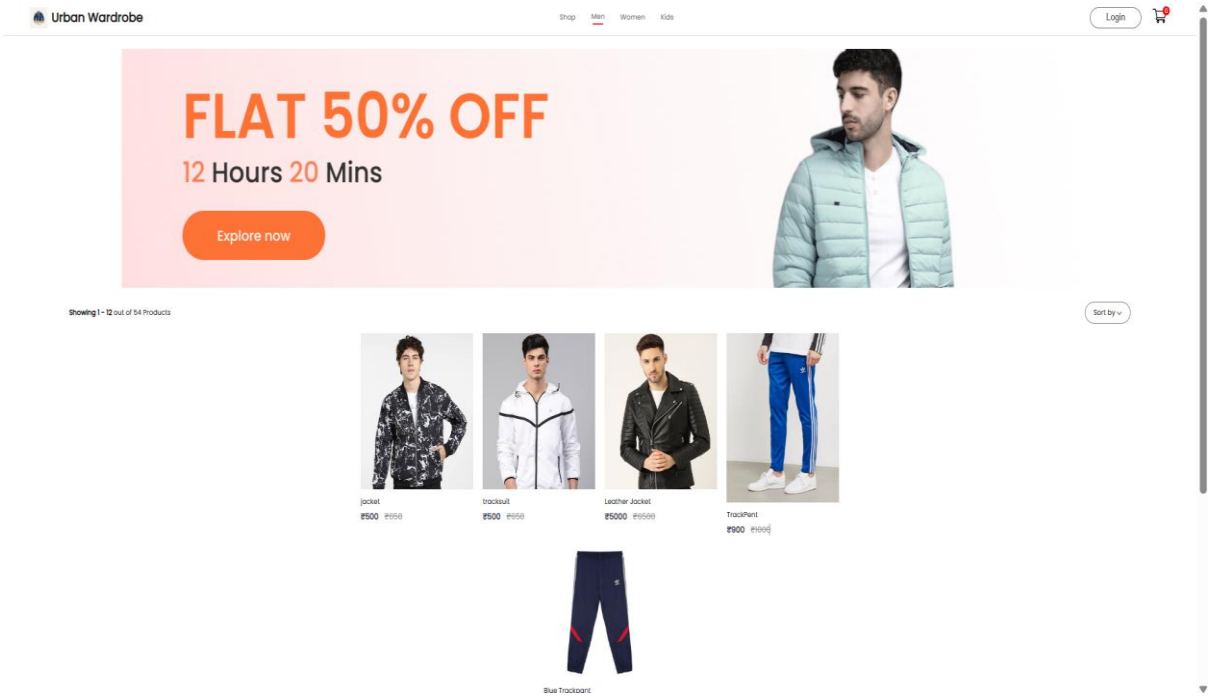
7.0 – Output

- Main Page:

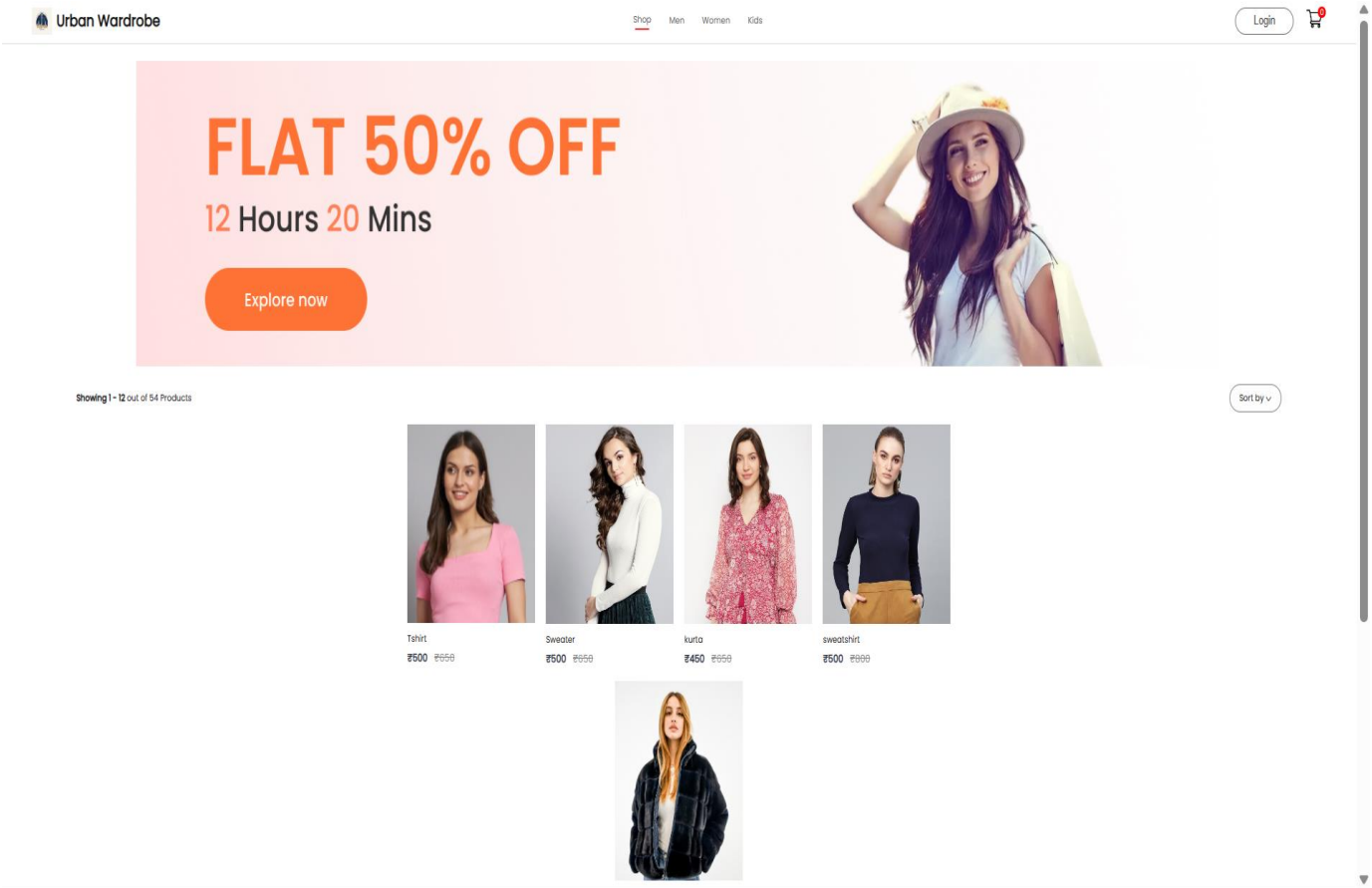




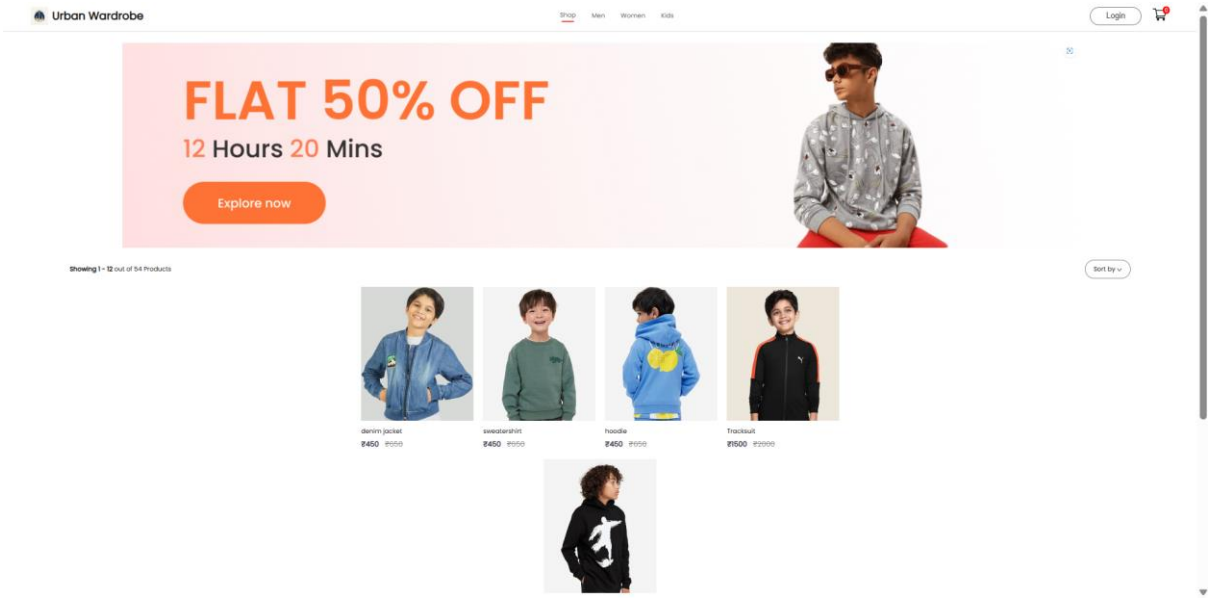
- Men Collection:

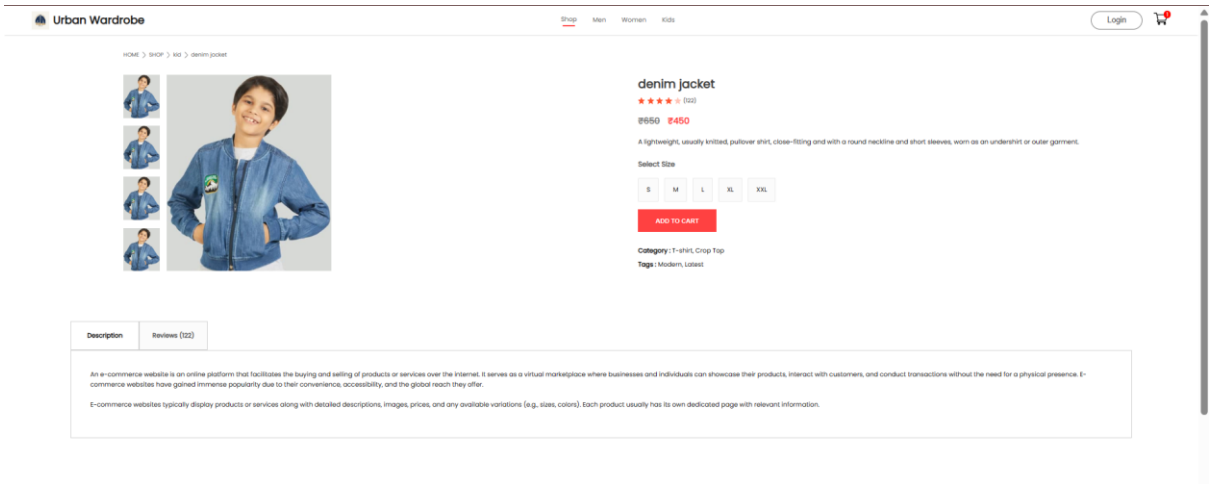


- Women Collection:

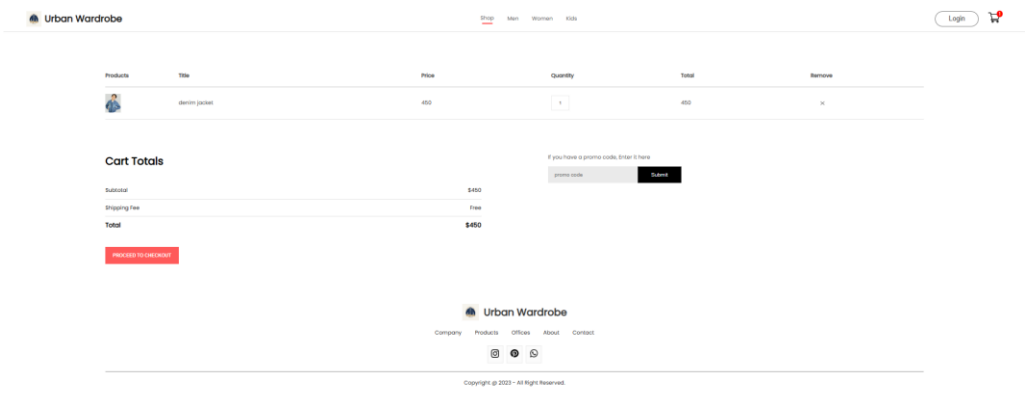


- Kids Collection:





- Cart:



8.0 - Conclusion

In conclusion, the development of the e-commerce website for Urban Wardrobe represents a significant milestone in providing a seamless online shopping experience for fashion enthusiasts. Through meticulous planning, robust backend architecture, and captivating frontend design, Urban Wardrobe has successfully positioned itself as a premier destination for fashion-forward individuals seeking the latest trends and styles.

The implementation of a three-tier architecture has enabled Urban Wardrobe to achieve a harmonious balance between presentation, functionality, and data management. The frontend, characterized by its intuitive user interface, responsive design, and immersive visual elements, captivates users from the moment they land on the website. Meanwhile, the backend, powered by cutting-edge technologies and scalable infrastructure, ensures smooth navigation, secure transactions, and efficient order processing.

Throughout the development process, Urban Wardrobe has prioritized not only the technical aspects of the website but also the user experience and accessibility. Rigorous testing methodologies, including functional, usability, compatibility, performance, and security testing, have been employed to guarantee a flawless browsing and shopping experience across devices and platforms.

Furthermore, Urban Wardrobe remains committed to continuous improvement and innovation, with plans for ongoing monitoring, maintenance, and optimization. By leveraging user feedback, analytics insights, and emerging technologies, Urban Wardrobe aims to enhance its offerings, streamline operations, and stay ahead of evolving customer expectations in the dynamic landscape of e-commerce.

In essence, the launch of the Urban Wardrobe e-commerce website signifies more than just a digital storefront; it symbolizes a commitment to style, quality, and customer satisfaction. With its seamless fusion of fashion and technology, Urban Wardrobe is poised to redefine online shopping and inspire confidence in every fashion-forward individual who steps into its virtual aisles.

8.1 - Future Modifications

Looking ahead, Urban Wardrobe recognizes the importance of staying at the forefront of e-commerce innovation to meet the evolving needs and preferences of its customers. Here are some potential future modifications and enhancements that Urban Wardrobe could consider implementing:

Personalization Features:

- Implement personalized product recommendations based on user preferences, browsing history, and purchase behaviour.
- Introduce virtual try-on capabilities using augmented reality (AR) technology to enhance the shopping experience and increase confidence in purchasing decisions.

Enhanced Mobile Experience:

- Optimize the website for mobile devices with a mobile-first approach, ensuring seamless navigation, faster load times, and intuitive user interactions on smartphones and tablets.
- Explore progressive web app (PWA) development to deliver app-like experiences, including offline access, push notifications, and home screen installation, improving user engagement and retention.

Social Commerce Integration:

- Integrate social media features and functionality, allowing users to share products, create wishlists, and receive recommendations from friends and influencers directly within the Urban Wardrobe platform.
- Enable social login options to streamline account creation and login processes, leveraging social media authentication for increased convenience and user acquisition.

Expanded Product Range:

- Diversify the product catalog to include a wider range of fashion items, accessories, and lifestyle products, catering to a broader audience and addressing niche market segments.
- Partner with emerging designers, brands, and influencers to offer exclusive collections and collaborations, driving excitement and differentiation in the market.

Voice Commerce Integration:

- Explore voice-enabled shopping experiences using voice assistants like Amazon Alexa or Google Assistant, allowing users to browse products, place orders, and track shipments using voice commands.
- Leverage natural language processing (NLP) and voice recognition technologies to deliver personalized recommendations and assistance, enhancing accessibility and convenience for users.

Subscription Services:

- Introduce subscription-based services such as curated fashion boxes, monthly memberships, or loyalty programs, offering customers exclusive perks, discounts, and early access to new arrivals.
- Implement recurring billing and subscription management features to automate subscription renewals, billing cycles, and customer communications, improving customer retention and lifetime value.

Enhanced Customer Support:

- Integrate live chat support, chatbots, and virtual assistants to provide real-time assistance, answer queries, and resolve customer issues more efficiently.
- Implement self-service features such as order tracking, returns management, and FAQs to empower customers to find solutions independently and reduce support ticket volume.

Blockchain Integration:

- Explore blockchain technology for supply chain transparency, product authentication, and counterfeit prevention, providing customers with confidence in the authenticity and origin of products.
- Implement blockchain-based payment solutions for secure, transparent, and decentralized transactions, offering enhanced privacy, fraud prevention, and lower transaction costs.

By embracing these future modifications and enhancements, Urban Wardrobe can strengthen its position as a leading e-commerce destination, driving customer engagement, loyalty, and satisfaction in the competitive fashion industry landscape. Through a commitment to innovation, customer-centricity, and continuous improvement, Urban Wardrobe can anticipate and adapt to emerging trends, technologies, and consumer preferences, ensuring long-term success and relevance in the ever-evolving digital marketplace.

9.0 - Appendix

- Frontend Code

Navbar.jsx:

```
import React, { useContext, useRef, useState } from 'react'
import './Navbar.css'
import { Link } from 'react-router-dom'
import logo from '../Assets/logo.png'
import cart_icon from '../Assets/cart_icon.png'
import { ShopContext } from '../../Context/ShopContext'
import nav_dropdown from '../Assets/nav_dropdown.png'

const Navbar = () => {

  let [menu,setMenu] = useState("shop");
  const {getTotalCartItems} = useContext(ShopContext);

  const menuRef = useRef();

  const dropdown_toggle = (e) => {
    menuRef.current.classList.toggle('nav-menu-visible');
    e.target.classList.toggle('open');
  }

  return (
    <div className='nav'>
      <Link to='/' onClick={()=>{setMenu("shop")}} style={{ textDecoration: 'none' }} className="nav-logo">
        <img src={logo} alt="logo" />
        <p>Urban Wardrobe</p>
      </Link>
      <img onClick={dropdown_toggle} className='nav-dropdown' src={nav_dropdown} alt="" />
      <ul ref={menuRef} className="nav-menu">
        <li onClick={()=>{setMenu("shop")}}><Link to='/' style={{ textDecoration: 'none'
}}>Shop</Link>{menu==="shop"?<hr/>:<></>}</li>
        <li onClick={()=>{setMenu("mens")}}><Link to='/mens' style={{ textDecoration: 'none'
}}>Men</Link>{menu==="mens"?<hr/>:<></>}</li>
        <li onClick={()=>{setMenu("womens")}}><Link to='/womens' style={{ textDecoration: 'none'
}}>Women</Link>{menu==="womens"?<hr/>:<></>}</li>
        <li onClick={()=>{setMenu("kids")}}><Link to='/kids' style={{ textDecoration: 'none'
}}>Kids</Link>{menu==="kids"?<hr/>:<></>}</li>
      </ul>
      <div className="nav-login-cart">
        {localStorage.getItem('auth-token')
        ?<button onClick={()=>{localStorage.removeItem('auth-token');window.location.replace("/");}}>Logout</button>
        :<Link to='/login' style={{ textDecoration: 'none' }}><button>Login</button></Link>}
        <Link to="/cart"><img src={cart_icon} alt="cart"/></Link>
        <div className="nav-cart-count">{getTotalCartItems()}</div>
      </div>
```

```
    </div>
  )
}

export default Navbar
```

- Backend Code

Index.js:

```
const port = 4000;
const express = require("express");
const app = express();
const mongoose = require("mongoose");
const jwt = require("jsonwebtoken");
const multer = require("multer");
const path = require("path");
const cors = require("cors");

app.use(express.json());
app.use(cors());

// Database Connection With MongoDB
mongoose.connect("mongodb+srv://dassanmol250:dyvyansh_mongoDB@cluster0.tpcxqwm.mongodb.net/e-commerce");
// paste your mongoDB Connection string above with password
// password should not contain '@' special character

//Image Storage Engine
const storage = multer.diskStorage({
    destination: './upload/images',
    filename: (req, file, cb) => {
      console.log(file);
        return cb(null, `${file.fieldname}_${Date.now()}${path.extname(file.originalname)}`)
    }
})
const upload = multer({storage: storage})
app.post("/upload", upload.single('product'), (req, res) => {
    res.json({
        success: 1,
        image_url: `http://localhost:4000/images/${req.file.filename}`
    })
})
app.use('/images', express.static('upload/images'));

// MiddleWare to fetch user from database
const fetchuser = async (req, res, next) => {
  const token = req.header("auth-token");
  if (!token) {
```

```
    res.status(401).send({ errors: "Please authenticate using a valid token" });
  }
  try {
    const data = jwt.verify(token, "secret_ecom");
    req.user = data.user;
    next();
  } catch (error) {
    res.status(401).send({ errors: "Please authenticate using a valid token" });
  }
};


// Schema for creating user model
const Users = mongoose.model("Users", {
  name: {
    type: String,
  },
  email: {
    type: String,
    unique: true,
  },
  password: {
    type: String,
  },
  cartData: {
    type: Object,
  },
  date: {
    type: Date,
    default: Date.now,
  },
});

// Schema for creating Product
const Product = mongoose.model("Product", {
  id: {
    type: Number,
    required: true,
  },
  name: {
    type: String,
    required: true,
  },
  image: {
    type: String,
    required: true,
  },
  category: {
    type: String,
```

```
    required: true,
  },
  new_price: {
    type: Number
  },
  old_price: {
    type: Number
  },
  date: {
    type: Date,
    default: Date.now,
  },
  avilable: {
    type: Boolean,
    default: true,
  },
});

app.get("/", (req, res) => {
  res.send("Root");
});

//Create an endpoint at ip/login for login the user and giving auth-token
app.post('/login', async (req, res) => {
  console.log("Login");
    let success = false;
    let user = await Users.findOne({ email: req.body.email });
    if (user) {
        const passCompare = req.body.password === user.password;
        if (passCompare) {
            const data = {
                user: {
                    id: user.id
                }
            }
      success = true;
      console.log(user.id);
      const token = jwt.sign(data, 'secret_ecom');
      res.json({ success, token });
        }
        else {
            return res.status(400).json({success: success, errors: "please try with correct email/password"})
        }
    }
    else {
        return res.status(400).json({success: success, errors: "please try with correct email/password"})
    }
})
```

```
//Create an endpoint at ip/auth for regestring the user in data base & sending token
app.post('/signup', async (req, res) => {
  console.log("Sign Up");
        let success = false;
        let check = await Users.findOne({ email: req.body.email });
        if (check) {
            return res.status(400).json({ success: success, errors: "existing user found with this email" });
        }
        let cart = {};
          for (let i = 0; i < 300; i++) {
          cart[i] = 0;
        }
        const user = new Users({
            name: req.body.username,
            email: req.body.email,
            password: req.body.password,
            cartData: cart,
        });
        await user.save();
        const data = {
            user: {
                id: user.id
            }
        }

        const token = jwt.sign(data, 'secret_ecom');
        success = true;
        res.json({ success, token })
    })

app.get("/allproducts", async (req, res) => {
  let products = await Product.find({});
  console.log("All Products");
    res.send(products);
});

app.get("/newcollections", async (req, res) => {
  let products = await Product.find({});
  let arr = products.slice(1).slice(-8);
  console.log("New Collections");
  res.send(arr);
});

app.get("/popularinwomen", async (req, res) => {
  let products = await Product.find({});
  let arr = products.splice(0,  4);
  console.log("Popular In Women");
```

```
  res.send(arr);
});

//Create an endpoint for saving the product in cart
app.post('/addtocart', fetchuser, async (req, res) => {
  console.log("Add Cart");
    let userData = await Users.findOne({_id:req.user.id});
    userData.cartData[req.body.itemId] += 1;
    await Users.findOneAndUpdate({_id:req.user.id}, {cartData:userData.cartData});
    res.send("Added")
  })

  //Create an endpoint for saving the product in cart
app.post('/removefromcart', fetchuser, async (req, res) => {
  console.log("Remove Cart");
    let userData = await Users.findOne({_id:req.user.id});
    if(userData.cartData[req.body.itemId]!=0)
    {
      userData.cartData[req.body.itemId] -= 1;
    }
    await Users.findOneAndUpdate({_id:req.user.id}, {cartData:userData.cartData});
    res.send("Removed");
  })

  //Create an endpoint for saving the product in cart
app.post('/getcart', fetchuser, async (req, res) => {
  console.log("Get Cart");
  let userData = await Users.findOne({_id:req.user.id});
  res.json(userData.cartData);

  })


app.post("/addproduct", async (req, res) => {
  let products = await Product.find({});
  let id;
  if (products.length>0) {
    let last_product_array = products.slice(-1);
    let last_product = last_product_array[0];
    id = last_product.id+1;
  }
  else
  { id = 1; }
  const product = new Product({
    id: id,
    name: req.body.name,
    image: req.body.image,
    category: req.body.category,
    new_price: req.body.new_price,
```

```
    old_price: req.body.old_price,
  });
  console.log(product);
  await product.save();
  console.log("Saved");
  res.json({success:true,name:req.body.name})
});

app.post("/removeproduct", async (req, res) => {
  const product = await Product.findOneAndDelete({ id: req.body.id });
  console.log("Removed");
  res.json({success:true,name:req.body.name})
});

app.listen(port, (error) => {
  if (!error) console.log("Server Running on port " + port);
  else console.log("Error : ", error);
});
```